

Django REST framework JWT Authentication — Social login — Login with Google

10 min read · Jul 8, 2024



Michal Drózdź

Follow



Listen



Share



Sign in with Google

Django REST Framework — Sign in with Google

Getting started

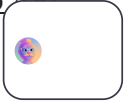
In this tutorial, you will learn how to log in with Google to your Django application, specifically how to use Google sign-in to create JWT authentication tokens in the Django REST framework.

This is the third part of the Django JWT Authentication series. If you are just getting started with JWT authentication in Django REST framework, make sure to check out the previous parts:



- Basic JWT auth setup: [Django REST API with JWT Authentication](#)

- User sign-up and email confirmation: [Django REST framework JWT Authentication Sign up with email confirmation](#)



Full code example is available in my GitHub repository:

- Github: <https://github.com/Mich0232/drf-jwt-social-tutorial>

Step 1 — Install Required Packages:

We will be using the `dj-rest-auth[with_social]` package to handle social authentication, such as login with Google.

Assuming you followed the previous parts of this guide, you can skip to [Step 2 — Social Authentication Setup](#).

Install requirements & basic setup

1. Start by installing `dj-rest-auth[with_social]` , as it ensures `django-allauth` is installed.

```
# requirements.txt
Django==5.0.6
djangorestframework==3.15.2
djangorestframework-simplejwt==5.3.1 # JWT token implementation for DRF
dj-rest-auth[with_social]==6.0.0     # REST API authentication utils
```

2. Add `django.contrib.sites`, `allauth`, `allauth.account`, and `dj_rest_auth.registration` apps to `INSTALLED_APPS` in your `django settings.py`:

```
INSTALLED_APPS = [
    ...,
    'django.contrib.sites',
    ...,
    'rest_framework',
    'rest_framework.authtoken',
    ...,
    'allauth',
    'allauth.account',
    'allauth.socialaccount',
    'dj_rest_auth.registration',
]

# django.contrib.sites
SITE_ID = 1
```



3. Add `allauth.account.middleware.AccountMiddleware` to `MIDDLEWARE` in `settings.py`

```
MIDDLEWARE = [
    ...
    "allauth.account.middleware.AccountMiddleware",
]
```



4. Email addresses coming from social authentication providers are considered verified, therefore, for the sake of this tutorial, we can disable email verification.

```
ACCOUNT_AUTHENTICATION_METHOD = "email" # Use Email / Password authentication
ACCOUNT_USERNAME_REQUIRED = False
ACCOUNT_EMAIL_REQUIRED = True
ACCOUNT_EMAIL_VERIFICATION = "none" # Do not require email confirmation
```

Full list of available settings: [Django Allauth Configuration](#)

5. Run Django migrations: `python manage.py migrate`

Step 2 — Social Authentication Setup

Load environment variables and update app configuration

1. We will be generating client ID and client secret to the Google API, therefore it's recommended to read those values from environment variables.

```
# requirements.txt
...
python-dotenv==1.0.1 # Load environment variables from .env file
```

2. Create `.env` file in the root of your project.

```
GOOGLE_OAUTH_CLIENT_ID=
GOOGLE_OAUTH_CLIENT_SECRET=
GOOGLE_OAUTH_CALLBACK_URL=
```

3. Load the `.env` file inside Django's `settings.py` and load the variables

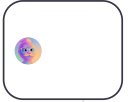
```
import os
from dotenv import load_dotenv

load_dotenv(".env")
```



```
...
```

```
# Google OAuth
GOOGLE_OAUTH_CLIENT_ID = os.getenv("GOOGLE_OAUTH_CLIENT_ID")
GOOGLE_OAUTH_CLIENT_SECRET = os.getenv("GOOGLE_OAUTH_CLIENT_SECRET")
GOOGLE_OAUTH_CALLBACK_URL = os.getenv("GOOGLE_OAUTH_CALLBACK_URL")
```



4. Add `allauth.socialaccount.provider.google` to `INSTALLED_APPS` in your Django `settings.py`

```
INSTALLED_APPS = [
    ...
    'allauth.socialaccount',
    'allauth.socialaccount.providers.google', # <- Support Login with Google
    ...
]
```

5. Add `allauth`'s social provider configuration.

```
# django-allauth (social)
# Authenticate if local account with this email address already exists
SOCIALACCOUNT_EMAIL_AUTHENTICATION = True
# Connect local account and social account if local account with that email address already exists
SOCIALACCOUNT_EMAIL_AUTHENTICATION_AUTO_CONNECT = True
SOCIALACCOUNT_PROVIDERS = {
    "google": {
        "APPS": [
            {
                "client_id": GOOGLE_OAUTH_CLIENT_ID,
                "secret": GOOGLE_OAUTH_CLIENT_SECRET,
                "key": "",
            },
        ],
        "SCOPE": ["profile", "email"],
        "AUTH_PARAMS": {
            "access_type": "online",
        },
    }
}
```

Step 3 — Create API endpoints

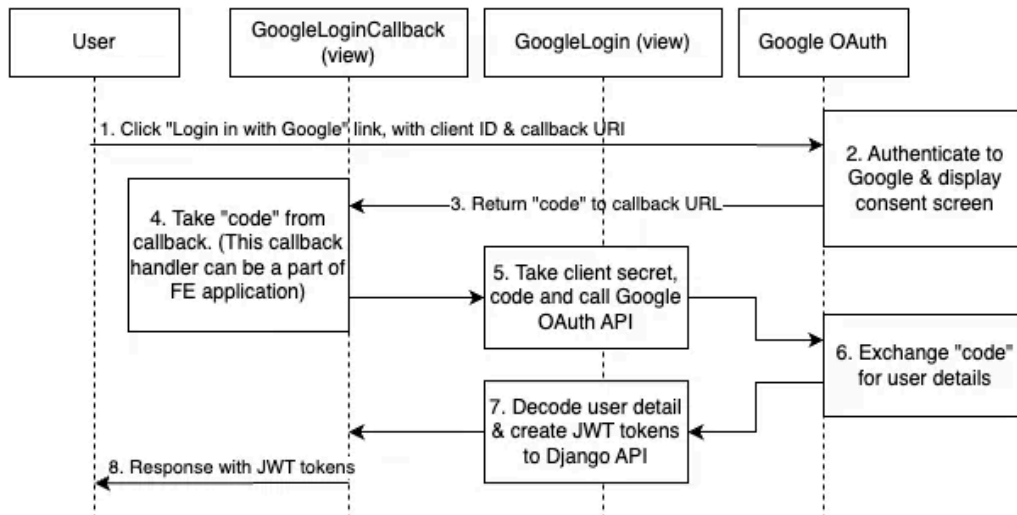
Social authentication process consists of a few API calls. Packages like `dj-rest-auth` provide views that handle most of that logic. We only need to expose the correct view depending on how we want to use it and what providers we are supporting.



There are several methods of exchanging data between the client and the OAuth provider called grant types. In this tutorial, we will be using the Authorization Code grant.

You can read more about grant types here: [OAuth Grant Types](#)

Authentication flow sequence diagram



Authorization code grant OAuth flow

1. To organise our code, let's create a new Django app.

```
python manage.py startapp accounts
```

2. Create a GoogleLogin view that will be using `django-rest-auth`'s components inside `accounts/views.py`

```
from allauth.socialaccount.providers.google.views import GoogleOAuth2Adapter
from allauth.socialaccount.providers.oauth2.client import OAuth2Client
from dj_rest_auth.registration.views import SocialLoginView
from django.conf import settings

class GoogleLogin(SocialLoginView):
    adapter_class = GoogleOAuth2Adapter
    callback_url = settings.GOOGLE_OAUTH_CALLBACK_URL
    client_class = OAuth2Client
```

3. Create a GoogleLoginCallback view.

```
...
from urllib.parse import urljoin

import requests
from django.urls import reverse
from rest_framework import status
from rest_framework.response import Response
from rest_framework.views import APIView
```





```
class GoogleLoginCallback(APIView):
    def get(self, request, *args, **kwargs):
        """
        If you are building a fullstack application (eq. with React app next to Django)
        you can place this endpoint in your frontend application to receive
        the JWT tokens there - and store them in the state
        """

        code = request.GET.get("code")

        if code is None:
            return Response(status=status.HTTP_400_BAD_REQUEST)

        # Remember to replace the localhost:8000 with the actual domain name before deployment
        token_endpoint_url = urljoin("http://localhost:8000", reverse("google_login"))
        response = requests.post(url=token_endpoint_url, data={"code": code})

        return Response(response.json(), status=status.HTTP_200_OK)
```

4. Create a simple login page.

```
from django.conf import settings
from django.shortcuts import render
from django.views import View

...

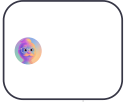
class LoginPage(View):
    def get(self, request, *args, **kwargs):
        return render(
            request,
            "pages/login.html",
            {
                "google_callback_uri": settings.GOOGLE_OAUTH_CALLBACK_URL,
                "google_client_id": settings.GOOGLE_OAUTH_CLIENT_ID,
            },
        )
```

login.html Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sign In</title>
  <style>
    body {
      width: 100vw;
      height: 100vh;
    }
  </style>

```





```
.center {
    height: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
}

.container {
    width: 400px;
    padding: 20px;
    border: 1px solid gray;
    border-radius: 4px;
}

.title {
    text-align: center;
}

</style>
</head>
<body>
<div class="center">
    <div class="container">
        <h1 class="title">Sign in</h1>
        <a href="https://accounts.google.com/o/oauth2/v2/auth?redirect_uri={{ google_callback_u
            Sign in with Google
        </a>
    </div>
</div>
</body>
</html>
```

5. Expose API endpoints in `urls.py` .

```
from django.contrib import admin
from django.urls import include, path, re_path

from accounts.views import GoogleLogin, GoogleLoginCallback, LoginPage

urlpatterns = [
    path("admin/", admin.site.urls),
    path("login/", LoginPage.as_view(), name="login"),
    path("api/v1/auth/", include("dj_rest_auth.urls")),
    re_path(r"^api/v1/auth/accounts/", include("allauth.urls")),
    path("api/v1/auth/registration/", include("dj_rest_auth.registration.urls")),
    path("api/v1/auth/google/", GoogleLogin.as_view(), name="google_login"),
    path(
        "api/v1/auth/google/callback/",
        GoogleLoginCallback.as_view(),
        name="google_login_callback",
    ),
]
```

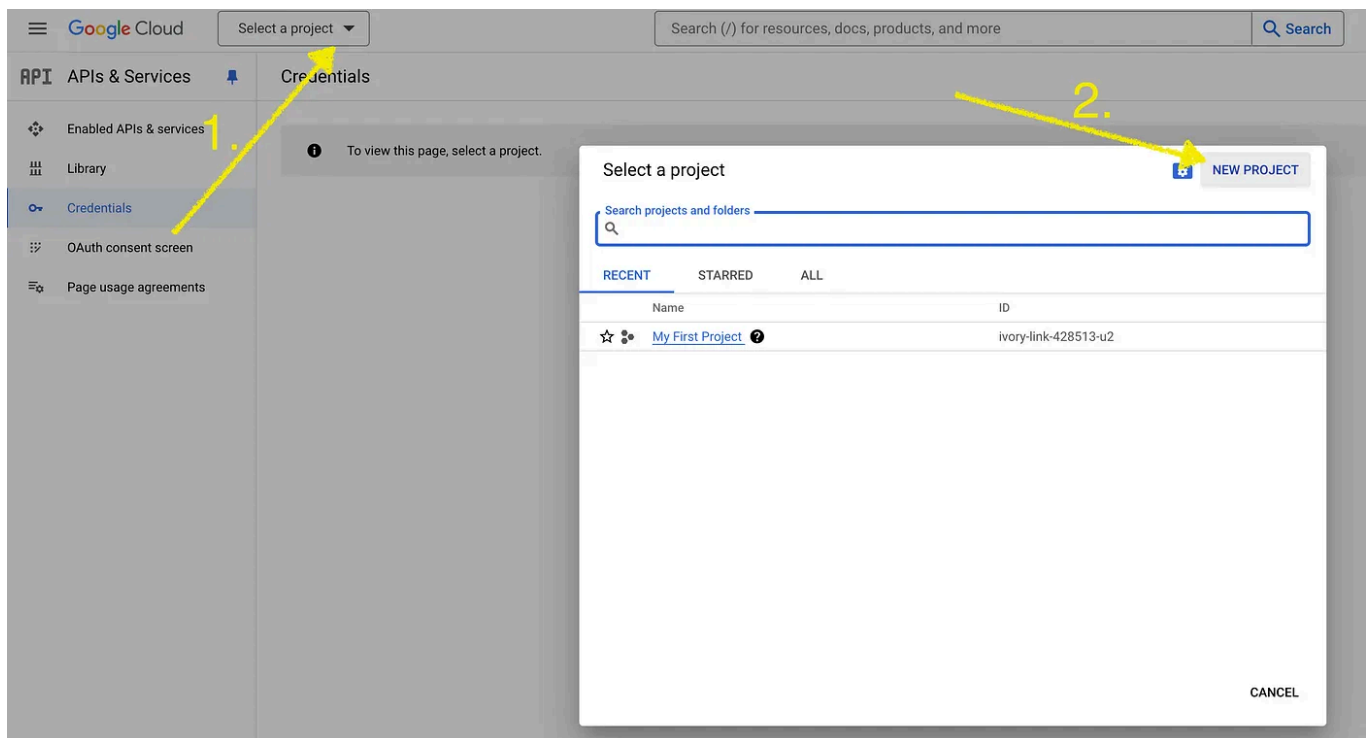


Step 4— Google OAuth application

Once we've scaffolded the API and backend setup, we can proceed to creating Google credentials.

1. Go to Google Cloud console API's dashboard: <https://console.cloud.google.com/apis/dashboard>

2. Select an existing project or create a new one.



Google Cloud — create new project

3. Make sure you've selected the project, then go to **OAuth consent screen** > select "External" > click "Create".



Google Cloud

drf-jwt-social-tutorial

Search (/) for

API APIs & Services

Enabled APIs & services

Library

Credentials

OAuth consent screen

Page usage agreements

OAuth consent screen

Choose how you want to configure and register your app, including your target users. You can only associate one app with your project.

User Type

☐ Internal

Only available to users within your organization. You will not need to submit your app for verification. [Learn more about user type](#)

☒ External

Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. [Learn more about user type](#)

CREATE

[Let us know what you think](#) about our OAuth experience

Google Cloud — create OAuth consent screen

4. Provide basic information about your app; most of it is optional.

Edit app registration

1 OAuth consent screen

2 Scopes

3 Test users

4 Summary

App information

This shows in the consent screen, and helps end users know who you are and contact you

App name *

My app

The name of the app asking for consent

Google Cloud — Create App Registration

5. Select email and profile scopes.

✕ Update selected scopes



Only scopes for enabled APIs are listed below. To add a missing scope to this screen, find and enable the API in the [Google API Library](#) or use the Pasted Scopes text box below. Refresh the page to see any new APIs you enable from the Library.

☰ Filter Enter property name or value



API	Scope	User-facing description
☒	.../auth/userinfo .email	See your primary Google Account email address
☒	.../auth/userinfo .profile	See your personal info, including any personal info you've made publicly available
☐	openid	Associate you with your personal info on Google

Google Cloud — select OAuth scopes

6. For testing purposes, you can use the email address assigned to your Google account or add testing users. If you need more test users, add them here.





✓ OAuth consent screen — ✓ Scopes — 3 **Test users** — 4 Summary

Test users

While publishing status is set to "Testing", only test users are able to access the app. Allowed user cap prior to app verification is 100, and is counted over the entire lifetime of the app. [Learn more](#)

+ ADD USERS

Filter Enter property name or value



User information

No rows to display

SAVE AND CONTINUE

CANCEL

Google Cloud — OAuth app test users

7. Go to "Summary" and click "Create".

Get Michal Dróždź's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

8. Switch to **Credentials** > click "Create Credentials" > "OAuth client ID".



Google Cloud drf-jwt-social-tutorial Search (/) for

APIs & Services

- Enabled APIs & services
- Library
- Credentials**
- OAuth consent screen
- Page usage agreements

Credentials + CREATE CREDENTIALS DELETE RESTORE DELETED CREDENTIALS

Create credentials to access your project

API Keys

☐	Name
No API keys to display	

OAuth 2.0 Client IDs

☐	Name	Creation date
No OAuth clients to display		

API key
Identifies your project using a simple API key to check quota and access

OAuth client ID
Requests user consent so your app can access the user's data

Service account
Enables server-to-server, app-level authentication using robot accounts

Help me choose
Asks a few questions to help you decide which type of credential to use

Google Cloud — create OAuth Client ID

9. Pick “Web application” > Give it a name > Select authorized callback domains -> this should match the URL of the GoogleLoginCallback view.





Create OAuth client ID



A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *

Web application



Name *

My web app

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.



The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins ?

For use with requests from a browser

+ ADD URI

Authorized redirect URIs ?

For use with requests from a web server

URIs 1 *

http://localhost:8000

URIs 2 *

http://localhost:8000/api/v1/auth/google/callback/

+ ADD URI



Note: It may take 5 minutes to a few hours for settings to take effect

CREATE

CANCEL

Google Cloud — OAuth client settings

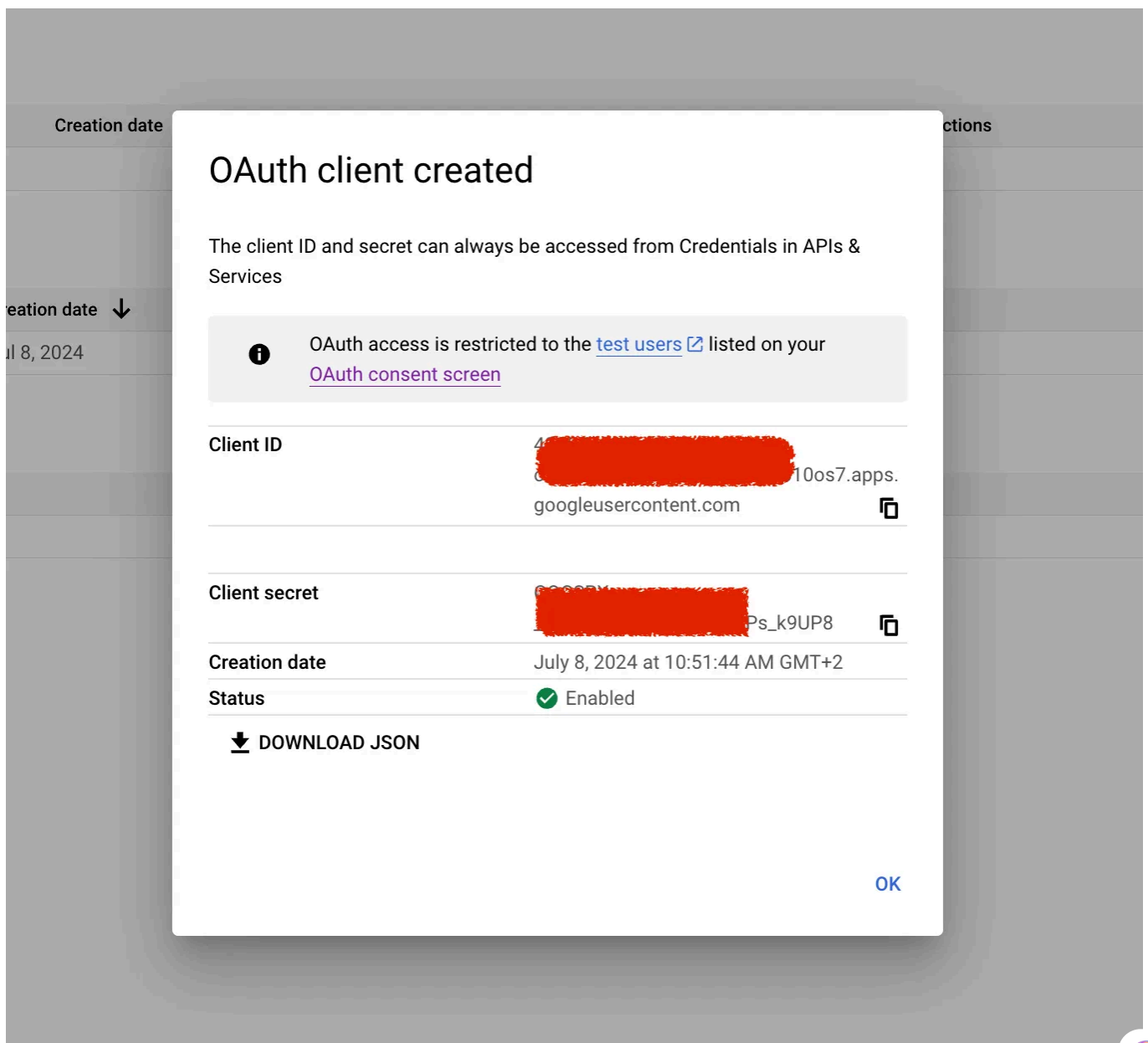


Side note:

I will go into details about frontend applications using JWT authentication later in this article, but:

*If you're using a frontend application like React running under a different domain, make sure to add it to **Authorized JavaScript origins** (e.g., `localhost:3000`). Also, if you exposed the callback endpoint as part of the React application, change the **Authorized redirect URIs** to your frontend application's domain.*

10. Click “Create” and save the OAuth app credentials.



Google Cloud — OAuth client credentials



Step 5— Testing the integration

1. Populate the `.env` file with your OAuth app credentials and restart the Django server.

```
GOOGLE_OAUTH_CLIENT_ID=<client_id>
GOOGLE_OAUTH_CLIENT_SECRET=<client_secret>
GOOGLE_OAUTH_CALLBACK_URL=http://localhost:8000/api/v1/auth/google/callback/
```



2. Go to the login page and click “Sign in with Google”.

Sign in

[Sign in with Google](#)

localhost:8000/login/

3. Sign in to your Google account.

Sign in with Google

Sign in

to continue to **My app**

[Forgot email?](#)

To continue, Google will share your name, email address, language preference, and profile picture with My app.

[Create account](#)

[Next](#)

English (United States) ▼

[Help](#)

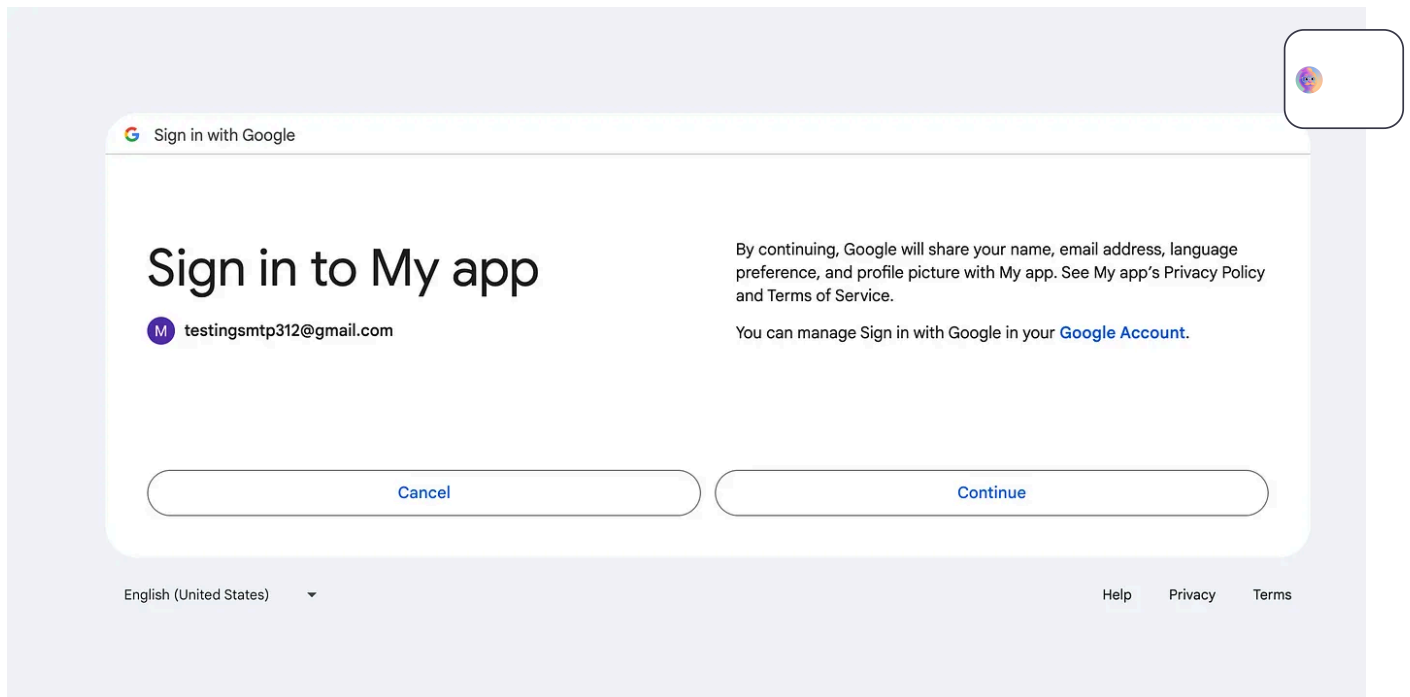
[Privacy](#)

[Terms](#)

Sign in with Google

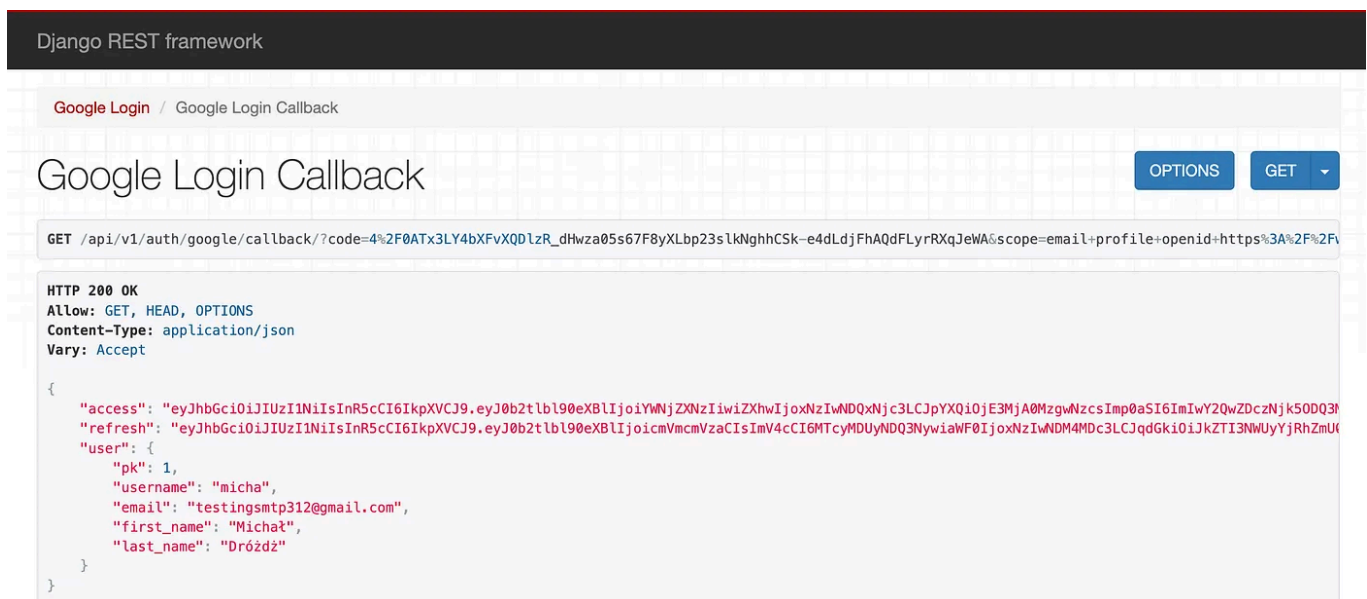


4. Click “Continue”.



Sign in with Google — consent screen

5. View access and refresh tokens.



GoogleLogin view — successfull response

Done! You can now use the tokens to authenticate against other APIs.

Step 6 — Understanding email addresses

When implementing social authentication, Django creates a separate model to hold the information about an account.

When we look at the Django admin, we can see that some models were created.





Site administration

ACCOUNTS

Email addresses

+ Add

 Change

AUTH TOKEN

Tokens

+ Add

 Change

AUTHENTICATION AND AUTHORIZATION

Groups

+ Add

 Change

Users

+ Add

 Change

SITES

Sites

+ Add

 Change

SOCIAL ACCOUNTS

Social accounts

+ Add

 Change

Social application tokens

+ Add

 Change

Social applications

+ Add

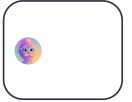
 Change

We have:

- Accounts / Email addresses — this comes from django-allauth
- Auth token / Tokens — this is a default Django REST framework token
- Groups & Users — standard stuff from Django auth framework
- Sites — Django Sites framework
- Social Accounts / Social Accounts; Social application tokens; Social applications — all from django-allauth



What is worth mentioning are these few things:



1. When a user logs in using a social provider, we create a Social Account for them. A Social Account ties the user with a provider (e.g., Google).
2. Users and Social Accounts can be tied together to allow multiple ways of authentication to a single account.
3. Social Applications allow you to set social provider settings via Django admin (instead of settings.py).
4. When you register a new user via the dj-rest-auth API, an EmailAddress instance is also created.
5. EmailAddresses are a feature allowing users to have multiple email addresses assigned to a single User.

Step 7— Using the Google login with frontend application (optional)

Let's assume you're using the Django app to serve the API and you have your frontend application that consumes it.

Here are a few steps you need to change in order to access social authentication from your frontend app.

This topic deserves a separate tutorial; therefore, I'll just quickly go through the required changes with a simple React app using ViteJS.

1. Render login button

```
import { googleCallbackUri, googleClientId } from './config.ts';

function App() {
  const googleSignInUrl = `https://accounts.google.com/o/oauth2/v2/auth?redirect_uri=${googleCa

  return (
    <div style={{ padding: 20 }}>
      <a href={googleSignInUrl}>Sign in with Google</a>
    </div>
  );
}

export default App;
```

config.ts

```
export const googleCallbackUri = "http://localhost:3000/google/callback/";
export const googleClientId = import.meta.env.VITE_GOOGLE_OAUTH_CLIENT_ID;
```



Django's .env

```
...
GOOGLE_OAUTH_CALLBACK_URL=http://localhost:3000/google/callback/
...
```



2. Google will respond with a callback request containing the `code` query parameter. Let's create a route that can handle this request.

For that matter, I'll use react-router-dom's loader to extract the code from the request and forward it to our API.

That's how it will look:

loader.ts

```
import './index.css';
import { redirect } from 'react-router-dom';

export const handleGoogleCallback = async ({ request }) => {
  // Exchange callback's code for JWT tokens

  const url = new URL(request.url);
  const code = url.searchParams.get('code');

  if (code) {
    try {
      const response = await fetch('http://localhost:8000/api/v1/auth/google/', {
        method: 'POST',
        body: JSON.stringify({ code }),
        headers: {
          'Content-Type': 'application/json',
        },
      });
    } catch (err) {
      console.error(err);
      throw new Response('Bad request', { status: 400 });
    }
  }

  const jwtData = await response.json();
  console.log(jwtData);

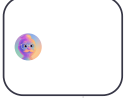
  return redirect('/');
}

throw new Response('Not Found', { status: 404 });
};
```



3. Let's change how we render our React app.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
```



```
import App from './App.tsx';
import './index.css';
import { createBrowserRouter, RouterProvider } from 'react-router-dom';
import { handleGoogleCallback } from './loader.ts';

const router = createBrowserRouter([
  {
    path: '/',
    element: <App />,
  },
  {
    path: '/google/callback',
    loader: handleGoogleCallback,
    element: <div>Loading...</div>,
  },
]);

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>,
);
```

App now supports two paths, the root one that renders the actual application as well as the `/google/callback` path that handles Google OAuth callbacks.

4. Making a request from `localhost:3000` to API on `localhost:8000` might raise a CORS exception issue. Use [django-cors-headers](#) to solve that.

- Install `django-cors-headers` via `pip`
- Add `corsheaders` to `INSTALLED_APPS`
- Add middleware

```
MIDDLEWARE = [
    ...
    "django.contrib.sessions.middleware.SessionMiddleware",
    "corsheaders.middleware.CorsMiddleware",
    "django.middleware.common.CommonMiddleware",
    ...
]
```

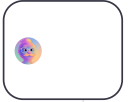
- Add CORS settings



```
# django-cors-headers
if DEBUG:
    CORS_ALLOW_ALL_ORIGINS = True
    CORS_ALLOW_CREDENTIALS = True
else:
```



```
CORS_ALLOWED_ORIGINS = [  
    "http://localhost:3000/",  
]
```



5. Lastly, we need to update our Google OAuth settings to allow requests from `localhost:3000` origin, as well as allow callback requests to `localhost:3000/google/callback/`

Authorized JavaScript origins

For use with requests from a browser

URIs 1 *

`http://localhost:3000`

[+ ADD URI](#)

Authorized redirect URIs

For use with requests from a web server

URIs 1 *

`http://localhost:8000`

URIs 2 *

`http://localhost:8000/api/v1/auth/google/callback/`

URIs 3 *

`http://localhost:3000/google/callback/`

[+ ADD URI](#)

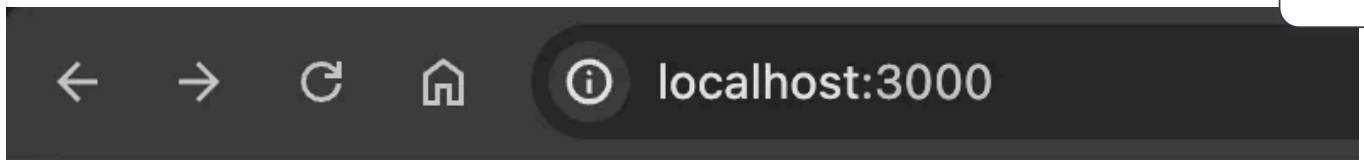
Note: It may take 5 minutes to a few hours for settings to take effect

SAVE

CANCEL



5. Test the integration > go to localhost:3000 > click “Sign in with Google” > authenticate.



[Sign in with Google](#)

localhost:3000

Inspect browser's console to check the token

```
▼ {access: 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1b190e...I6MX0.ro6c8RIHngxCvEULMmx...b2t1b190e...i0jF9.x-22mGyQUKItN_FX60LahIvH3kSf5yoV2Z9H_k-gAfY', user: {...}} ⓘ
  access: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1b190eXBlijo...iYWNjZXNzIiwiZXhwIj...
  refresh: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1b190eXBlijo...icmVmcmVzaCI6ImV4d...
  ▼ user:
    email: "testingsmtp312@gmail.com"
    first_name: "Michał"
    last_name: "Drózdź"
    pk: 1
    username: "micha"
    ▶ [[Prototype]]: Object
    ▶ [[Prototype]]: Object
>
```

Browser console

Conclusion ★

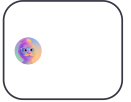
In this tutorial, we used `django-rest-auth` to set up social authentication in our Django REST application. We used the Google provider with the authorization code grant and implemented the flow using the backend as well as a simple React application.



Code Repository

Example code from this tutorial is available in my GitHub repository:

<https://github.com/Mich0232/drf-jwt-social-tutorial>



Further reading

If you found this tutorial helpful, you can **subscribe** to receive a **notification** for more Django tutorials! 🙌

Please make sure you check out the other parts of this tutorial:

- First part — the basic setup: [Django REST API with JWT Authentication](#)
- Second part: [Django REST framework JWT Authentication Sign up API with email confirmation](#)

Thanks for reading!

Michał

Django

Django Rest Framework

Social Authentication

Jwt

Google



Follow

Written by Michał Dróżdż

88 followers · 2 following

Fullstack Software Engineer; AWS Cloud Developer <https://github.com/Mich0232>

Responses (3)



Write a response

What are your thoughts?



Musikdady
Nov 25, 2024



Hey i copied your code and did exactly same

but im being returned

```
{  
  "key": "38849bfa21f75651979a5bb3ff18821fc9074841"  
}
```

and not acces and refresh tokens why so ?... [more](#)

 1  1 reply [Reply](#)

 Adam Ferrante
Mar 23

For those experiencing with Google authentication after following this guide as of 3-22-25, you may find a solution here:
<https://github.com/iMerica/dj-rest-auth/issues/673>

  1 reply [Reply](#)

 Meibrohimov
Jan 6

JSONDecodeError at /api/v1/auth/google/callback/Expecting value: line 1 column 1 (char 0)Request Method:GET

  1 reply [Reply](#)

More from Michal Dróždź



 Michal Dróždź

Setting Up a Django API with Django REST Framework (DRF): A Beginner's Guide

Learn how to create a REST API with Python and Django—tutorial



Jan 9 17 2

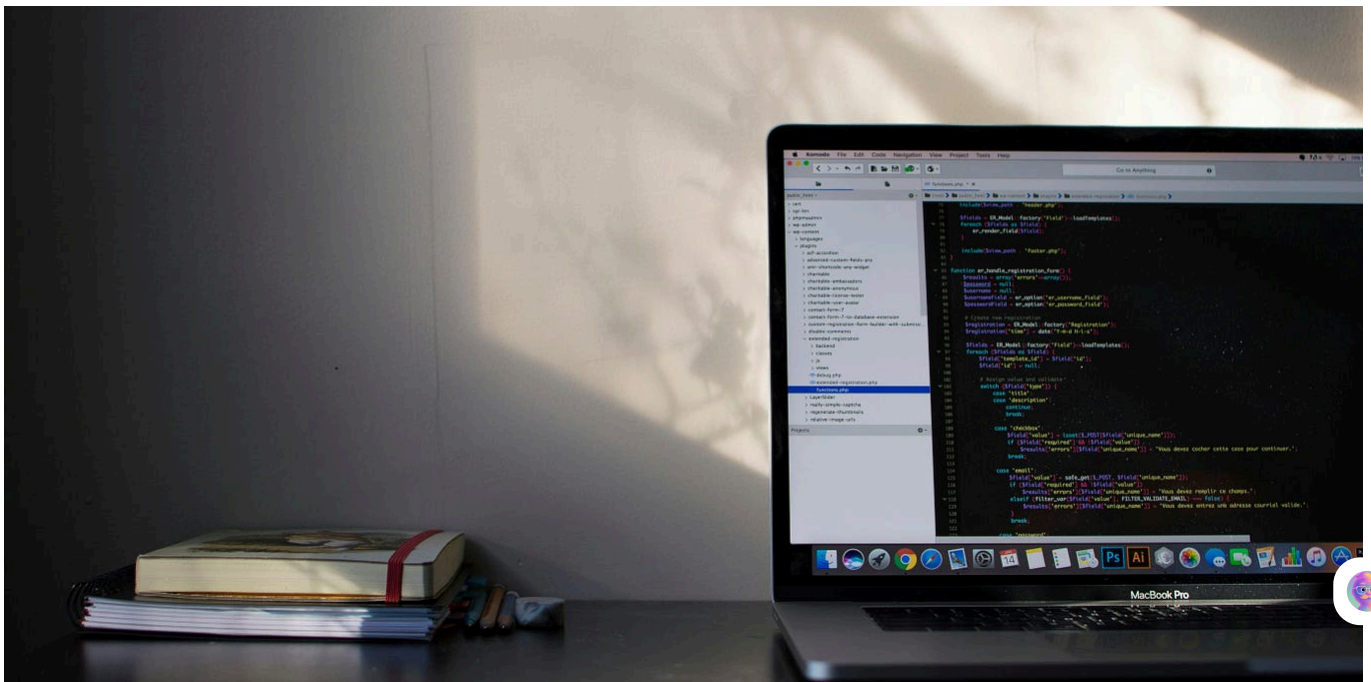


Michal Drózd

How to Reduce the Size of Your Docker Image and Build Time by 90%?

Reduce the size of your Docker image and speed-up the builds, by focusing on layer ordering, multi-stage builds, and Alpine images.

Jan 8 128 1

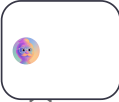


Michal Drózd

Django REST APIs with JWT Authentication using dj-rest-auth

How to enable JWT Authentication in your Django REST API?

Jul 4, 2024 71 2



 Michal Drózdź

Django REST framework JWT Authentication Sign up API with email confirmation.

How to register users over API using JWT authentication in Django REST framework?

Jul 5, 2024 97 1



See all from Michal Drózdź

Recommended from Medium





 Am Issath

JWT-Powered Django APIs: The Authentication Architecture Behind Real-World Products

No more half-baked auth systems. Learn how to build the exact JWT architecture real-world Django apps use to scale securely—from login to...

★ Jun 29 🖱 4



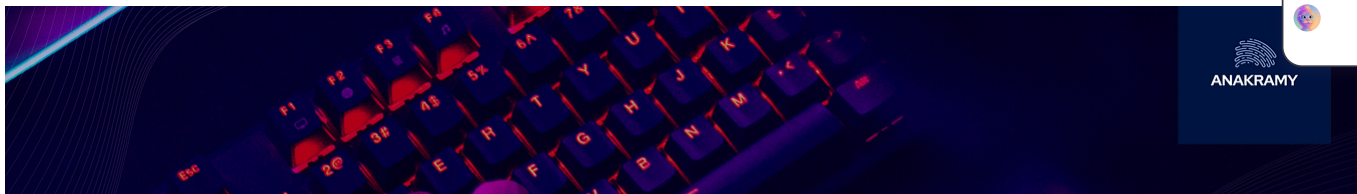
 In Python in Plain English by Gulsaba Fiha



How to Build a Multi-Tenant SaaS App with Django (2025 Complete Guide)

Are you building a SaaS application using Django and wondering how to support multiple clients or companies each with their own data...

6d ago 🖱️ 3



 Santhosh Adiga U

Hacking the HTTP Requests: A Deep Dive into Exploiting Web Applications

🌟 Jan 25 🖱️ 75 💬 2



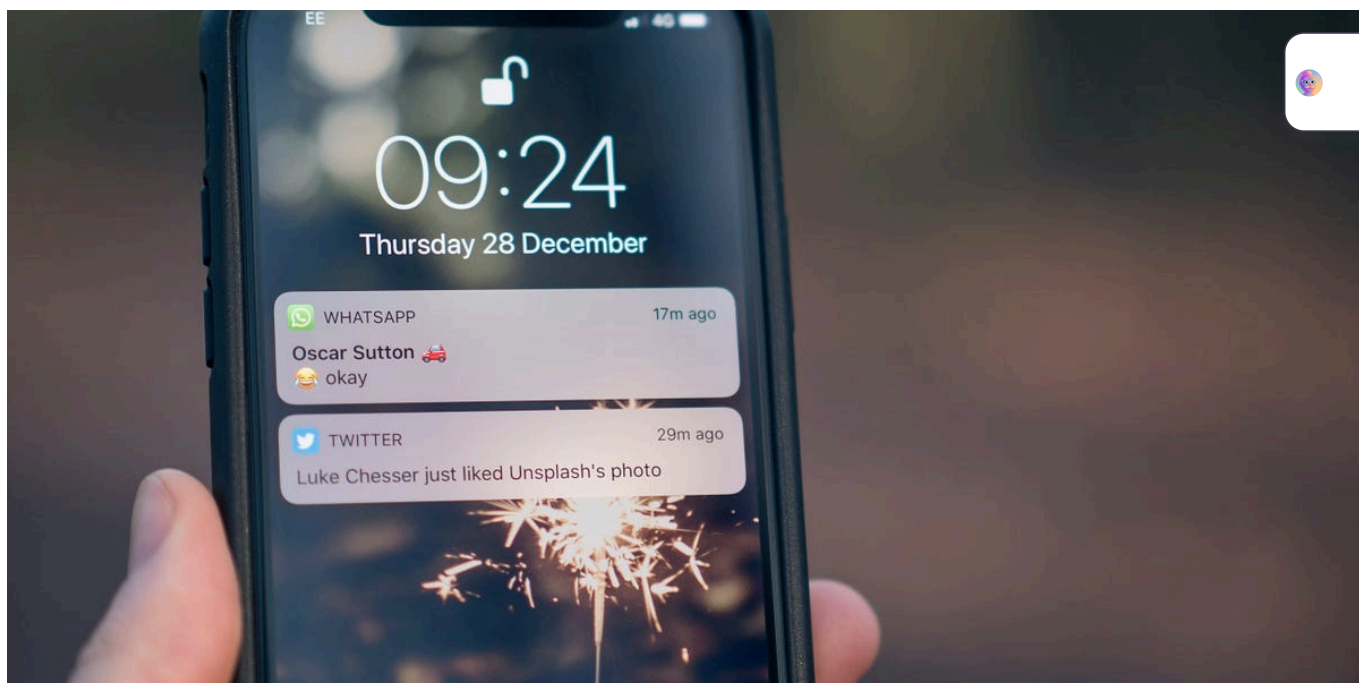
 Shahsad KP

No Middleware Needed: Understanding DRF's Per-View Strategy

Understanding DRF's Per-View Strategy for Flexible, Per-Endpoint API Control

Jul 6 🖱️ 2





 Priyanshu Rajput

How to Build a Push Notification System Using Django

Send Alerts, Updates & Messages Like a Pro (Web + Mobile Ready)

★ Jul 7 👏 41 💬 1



FASTAPI. HERE'S WHAT I LEARNED

 Nikulsinh Rajput

I Ditched Django for FastAPI. Here's What I Learned

From batteries-included to lightning-fast APIs with modern Python.

★ Jun 21 👏 29 💬 1



See more recommendations

